

PHẦN I - GIỚI THIỆU MỤC TIÊU VÀ PHẠM VI THỰC HIỆN

1. MỤC TIÊU THỰC HIỆN

Dự án được xây dựng nhằm củng cố các kỹ năng phát triển phần mềm nâng cao thông qua việc hiện thực hóa một hệ thống đấu giá trực tuyến (Bidding System) lấy cảm hứng từ eBay, hướng tới các mục tiêu cốt lõi sau:

Về mặt học thuật và kỹ năng lập trình:

- Củng cố tư duy hướng đối tượng (OOP):** Áp dụng triệt để 4 tính chất (Đóng gói, Kế thừa, Đa hình, Trừu tượng) trong thiết kế hệ thống DTO, DAO, Model và Controller giúp cấu trúc mã nguồn sạch và dễ bảo trì.
- Kiến trúc đa mô-đun (Multi-module):** Tổ chức dự án bằng Maven với các mô-đun tách biệt rõ ràng (client, server, common), phân định tường minh nhiệm vụ và tối ưu khả năng tái sử dụng mã nguồn.
- Lập trình mạng Socket:** Làm quen và triển khai lập trình Socket trong Java nhằm xây dựng cơ chế giao tiếp hai chiều thời gian thực (real-time) hiệu năng cao giữa hệ thống Client và Server.
- Tương tác dữ liệu đám mây:** Sử dụng kết nối JDBC thuần để truy xuất và quản lý cơ sở dữ liệu quan hệ MySQL được triển khai trực tiếp trên dịch vụ đám mây AWS RDS.
- Giao diện JavaFX hiện đại:** Xây dựng giao diện đồ họa (GUI) động, trực quan bằng JavaFX và FXML thay thế hoàn toàn cho giao diện dòng lệnh (CLI) truyền thống.

Về mặt nghiệp vụ hệ thống:

- Hiện thực hóa một sàn đấu giá trực tuyến hỗ trợ cơ chế tự động nâng giá và cập nhật trạng thái phiên chính xác.
- Đảm bảo quy trình đấu giá diễn ra minh bạch, an toàn, tuân thủ đúng quy tắc (bước giá tối thiểu, thời gian cấu hình).

Tích hợp Trí tuệ nhân tạo (AI Integration): Hệ thống ứng dụng thư viện **Langchain4j** kết nối với mô hình ngôn ngữ lớn (LLM), hỗ trợ xử lý đa phương thức gồm văn bản và ảnh Base64 sản phẩm, đem lại trải nghiệm tương tác thông minh cho người dùng.

2. PHẠM VI THỰC HIỆN

Dựa trên cấu trúc kiến trúc và hệ thống lớp hiện tại, phạm vi triển khai của hệ thống được xác định cụ thể như sau:

A. Phạm vi Kiến trúc & Công nghệ

- Mô hình:** Kiến trúc Client-Server. Client sử dụng JavaFX làm ứng dụng Desktop; Server chạy chương trình Socket đa luồng (ClientHandler kết hợp Thread Pool) phục vụ kết nối song song.
- Giao thức:** Custom Application Protocol trên nền TCP Socket, truyền nhận qua JSON. Hỗ trợ phát sự kiện (Broadcast Events) như thêm mới, sửa, xóa phiên đến toàn bộ client đang kết nối.
- Lưu trữ:** RDBMS MySQL đặt trên AWS RDS Cloud, thực hiện truy xuất qua mẫu thiết kế DAO và JDBC giúp quản lý kết nối hiệu quả.
- Tích hợp AI:** Giao tiếp trực tiếp qua socket bằng hành động CHAT_AI. Server làm trung gian gọi đến API của mô hình ngôn ngữ lớn để trả câu trả lời.

B. Phạm vi Chức năng (Phân quyền hệ thống)

Phân hệ Người mua (Bidder)

- Đăng ký, đăng nhập tài khoản và cập nhật thông tin cá nhân.
- Tìm kiếm, lọc (theo danh mục, trạng thái active/closed) và sắp xếp.
- Xem chi tiết sản phẩm (mô tả, ảnh, lịch sử giá, đồng hồ đếm ngược).
- Đặt giá (Place Bid) real-time và tự động kiểm tra tính hợp lệ.
- Tương tác với trợ lý AI để tư vấn sản phẩm qua ảnh chụp và câu hỏi.

Phân hệ Người bán (Seller)

- Tạo phiên đấu giá (tên, mô tả, ảnh Base64, giá sàn, bước giá, thời gian).
- Theo dõi danh sách phiên sở hữu và lịch sử đặt giá của khách hàng.
- Chỉnh sửa hoặc hủy bỏ phiên đấu giá trước mốc thời gian bắt đầu.

Phân hệ Quản trị viên (Admin)

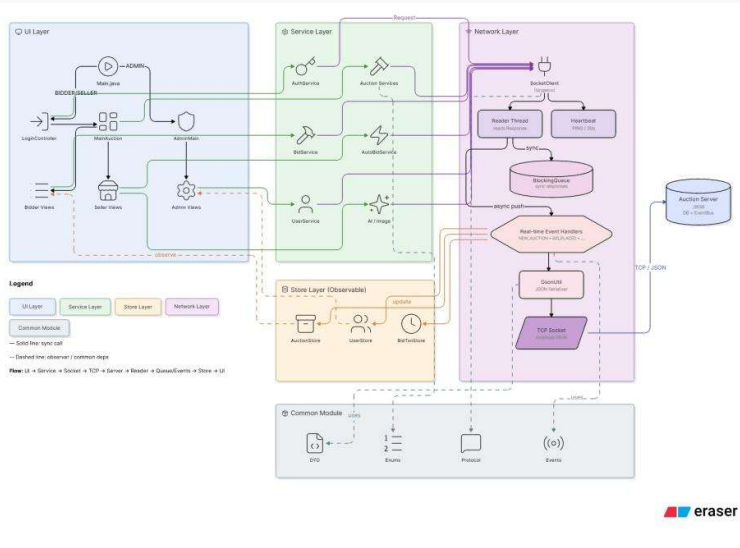
- Quản lý người dùng (Xem, thêm mới, cập nhật hoặc vô hiệu hóa tài khoản).
- Giám sát toàn diện hệ thống, có quyền hủy bỏ hoặc đóng sớm các phiên đấu giá vi phạm quy định.

PHẦN II - SƠ ĐỒ VÀ MÔ TẢ KIẾN TRÚC HỆ THỐNG

1.KIẾN TRÚC PHÂN TẦNG CLIENT

Đặc tả mô hình Phân tầng & Hướng sự kiện (Layered & Event-driven)

Chu trình luồng dữ liệu khép kín (Closed-loop Data Flow): UI Layer → Service Layer → Network Layer → Store Layer → UI Layer



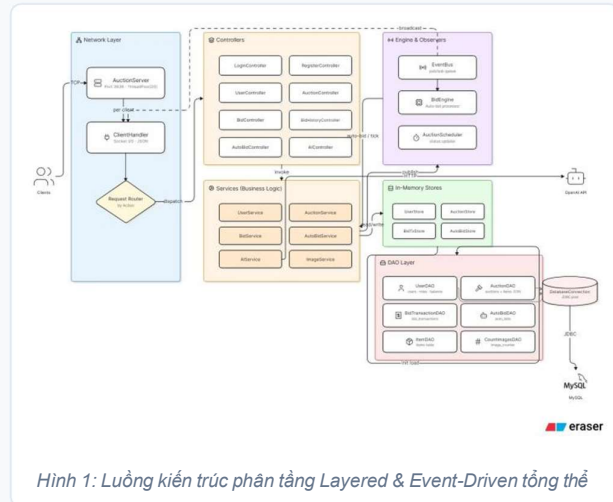
*ĐẶC TẢ CHI TIẾT CẤU TRÚC PHÂN TẦNG

Phân Tầng	Chức Năng Nghiệp Vụ Thực Thi	Cơ Chế Giao Tiếp & Liên Kết
UI Layer	LoginController phân quyền: Nhánh ADMIN qua AdminMain thành Admin Views; Nhánh BIDDER/SELLER qua MainAuction thành Bidder/Seller Views.	Đồng bộ (Sync Call) UI gọi hàm trực tiếp sang tầng Service để xử lý logic nghiệp vụ.
Service Layer	Điều hướng nghiệp vụ lõi: Xác thực (AuthService), Đấu giá (AuctionServices), Đặt lệnh thầu (BidService), Tự động đấu giá (AutoBidService), Quản lý user phối hợp module AI / Image.	Đồng bộ (Sync Call) Gọi trực tiếp đến SocketClient (Singleton). Dữ liệu được mã hóa JSON qua tiện ích GsonUtil.
Network Layer	Duy trì kết nối mạng TCP Socket với Server (Port 3636). Tích hợp luồng Heartbeat tự động thực hiện cơ chế PING mỗi 30 giây để giữ liên lạc ổn định.	Bất đồng bộ (Async Push) Phản hồi đồng bộ qua Reader Thread đẩy vào BlockingQueue; Luồng bất đồng bộ qua Event Handlers update trực tiếp xuống Store.
Store Layer	Quản lý và lưu trữ trạng thái cục bộ tại Client thông qua các kho lưu trữ dữ liệu chuyên biệt bao gồm: AuctionStore, UserStore, và BidTxnStore.	Mô hình Observer Khi Store được update dữ liệu từ Network, nó tự động phát tín hiệu (observe) kích hoạt các View cập nhật UI realtime.
Common Module	Định nghĩa tập trung các thành phần dùng chung toàn cục bao gồm: cấu trúc dữ liệu chuyển đổi DTO, danh mục chuẩn Enums, khung truyền tải Protocol, và hệ thống Events.	Một chiều (Uses) Tất cả các phân tầng trên đều kết nối phụ thuộc một chiều về module này để bảo đảm tính đồng nhất cấu trúc.

*ĐÁNH GIÁ ƯU ĐIỂM KHOA HỌC TRONG THIẾT KẾ

Độc lập & Dễ bảo trì (Separation of Concerns): Phân ranh giới rõ ràng giữa giao diện, logic và tầng dữ liệu mạng giúp kỹ sư dễ dàng nâng cấp hiển thị giao diện hiển thị mà không ảnh hưởng tới lõi xử lý bên dưới.	Hiệu năng mượt mà (Non-blocking UI): Cơ chế nhận dữ liệu bất đồng bộ tách luồng kết hợp mẫu thiết kế Observer giúp giao diện luôn mượt mà, loại bỏ hoàn toàn hiện tượng đóng băng ứng dụng khi có lượng lớn dữ liệu đấu giá đổ về.
--	---

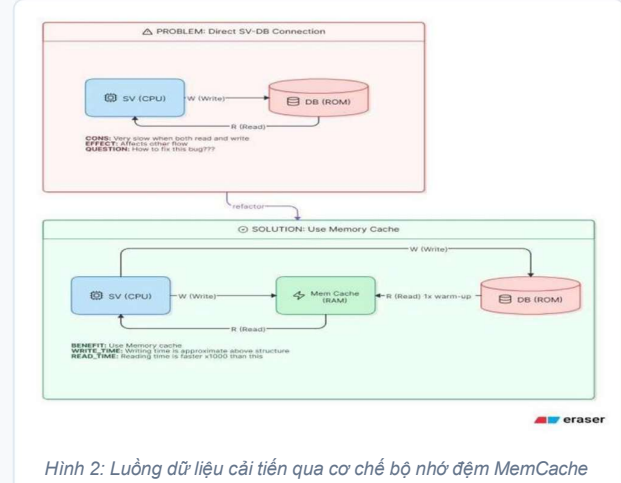
2. KIẾN TRÚC PHẦN TẦNG SERVER



Hệ thống được vận hành trên sự kết hợp chặt chẽ giữa kiến trúc phân tầng **Layered** và mô hình hướng sự kiện **Event-driven**, tạo nên một luồng xử lý bất đồng bộ khép kín, tối ưu hiệu năng từ lúc tiếp nhận đến khi lưu trữ:

- **Tầng Mạng (Network Layer) → Controllers:** Thành phần `AuctionServer` (Port 3636, ThreadPool 20) đón nhận các kết nối TCP từ phía Client, phân phối dữ liệu JSON cho `ClientHandler` dịch mã.
`Request Router` đóng vai trò điều phối (dispatch) chính xác yêu cầu tới Controllers.
- **Controllers → Services:** Bộ điều khiển chức năng nhận tham số mạng và kích hoạt trực tiếp (invoke) logic tương ứng ở tầng Services.
- **Services → In-Memory Stores & Bên ngoài:** Tầng Services xử lý logic nghiệp vụ, thực hiện tương tác hai chiều (read/write) với kho dữ liệu tạm `In-Memory Stores` để tối ưu tốc độ phản hồi. Đồng thời, `AIService` kết nối ngoại vi với OpenAI API qua HTTP.
- **Engine & Observers (Tự động hóa):** Hệ thống tích hợp khối tự động hóa gồm `EventBus` (Pub/Sub Queue) phối hợp với `BidEngine` và `AuctionScheduler`. Luồng tự động đặt giá (auto-bid/tick) liên kết ngược lại Services và phát tín hiệu truyền thông (broadcast) về `ClientHandler` để đẩy dữ liệu thời gian thực tới Client.
- **Stores / DAO Layer → MySQL:** Khi khởi động, tầng DAO Layer nạp dữ liệu cấu hình ban đầu (init load) lên bộ nhớ tạm. Trong quá trình vận hành, dữ liệu cập nhật xuống DAO và lưu vĩnh viễn vào `MySQL` thông qua kết nối tối ưu `JDBC Pool`.

3. CHIẾN LƯỢC TỐI ƯU HÓA BỘ NHỚ ĐỆM



⚠ VẤN ĐỀ HỆ THỐNG (DIRECT CONNECTION)

Ban đầu, máy chủ SV (CPU) thực hiện cả hai thao tác đọc R (Read) và ghi W (Write) trực tiếp với cơ sở dữ liệu gốc DB (ROM). Cơ chế này gây ra nhược điểm hệ thống bị chậm nghiêm trọng khi xảy ra đồng thời cả hai tác vụ đọc/ghi, ảnh hưởng trực tiếp đến các luồng xử lý khác.

✅ GIẢI PHÁP TỐI ƯU (MEMORY CACHE)

Hệ thống được cải tiến bằng cách chèn thêm một lớp bộ nhớ đệm **Mem Cache (RAM)** nằm giữa Máy chủ và Cơ sở dữ liệu để cô lập và phân tách luồng truy cập dữ liệu.

Cơ chế liên kết và luồng dữ liệu mới:

- **Thao tác Ghi (Write):** Khi xuất hiện dữ liệu mới, máy chủ đồng thời thực hiện hai luồng ghi: một luồng ghi vào bộ nhớ đệm `Mem Cache (RAM)` và một luồng ghi trực tiếp xuống `DB (ROM)` để đảm bảo tính lưu trữ vĩnh viễn.
- **Thao tác Đọc (Read):** Thay vì đọc dữ liệu từ DB thì SV đọc dữ liệu thông qua `Mem Cache (RAM)` giúp tăng hiệu suất và giảm áp lực cho Database vì đọc một lần duy nhất khi khởi động (`1x warm-up`) để chuẩn bị sẵn dữ liệu lên RAM.
- **Hiệu quả đạt được (Benefit):** Nhờ tốc độ đọc dữ liệu từ RAM nhanh hơn gấp **1000 lần** so với đọc trực tiếp ROM (`READ_TIME: faster x1000`), giải pháp giúp giải quyết tình trạng thất nút cổ chai và tối ưu hóa hiệu năng toàn hệ thống.

PHẦN III - CÁC CHỨC NĂNG ĐÃ ĐẠT ĐƯỢC

1. Thiết kế lớp, cây kế thừa và Design Pattern

Chức năng	Triển khai hoàn thiện các thực thể lõi: User, Item, Auction, BidTransaction . Vai trò người dùng (Bidder, Seller, Admin) tổ chức linh hoạt qua Enum <i>UserRole</i> gắn với thực thể User. Áp dụng chuẩn mực các nguyên tắc OOP và các Design Pattern kinh điển.
Giải pháp & Lý do	• OOP (Kế thừa & Đóng gói): Tạo lớp trừu tượng cơ sở <i>Entity</i> chứa id cho các model kế thừa. Lớp <i>Item</i> là cha của các phân loại cụ thể như <i>Art, Electronics, Vehicle</i>. Thuộc tính quan trọng (số dư <i>availableBalance</i>) được đóng gói (private/protected), chỉ sửa đổi qua phương thức kiểm soát rủi ro (<i>deposit, withdraw, freezeBalance</i>) để tránh thay đổi dữ liệu tùy tiện. • DAO Pattern: Toàn bộ thao tác cơ sở dữ liệu được tách biệt hoàn toàn vào các lớp DAO (như <i>UserDAO, AuctionDAO, BidTransactionDAO</i>). Giúp logic nghiệp vụ (Service/Engine) không dính chặt với các câu lệnh SQL, tuân thủ nguyên tắc Single Responsibility, dễ bảo trì hoặc đổi cơ sở dữ liệu sau này. • Observer Pattern: Ứng dụng mạnh mẽ qua package <i>observer</i> (với <i>EventBus, ClientNotificationObserver</i>) nhằm lắng nghe và phân phát sự kiện đấu giá, giảm thiểu sự phụ thuộc cứng (tight-coupling) giữa các thành phần.

2. Chức năng chính

Chức năng	Quản lý toàn diện vòng đời người dùng (đăng ký, đăng nhập, nạp/rút tiền, phân quyền). Quản lý sản phẩm và phiên đấu giá (tạo mới, bắt đầu, kết thúc). Xử lý nghiệp vụ đặt giá (Bid) và kiểm soát lỗi/ngoại lệ chặt chẽ.
Giải pháp & Lý do	• Kiểm soát nghiệp vụ tập trung: Nghiệp vụ đấu giá được giao cho <i>BidService</i> và <i>BidEngine</i> xử lý tập trung. Số tiền đặt cọc của Bidder đang dẫn đầu được hệ thống đóng băng ngay lập tức (<i>frozenBalance</i>) nhằm đảm bảo tính thanh khoản, và tự động hoàn trả (<i>unfreezeBalance</i>) khi có người khác trả giá cao hơn. • Xử lý ngoại lệ chuyên biệt: Xây dựng package <i>exception</i> riêng biệt (ví dụ: <i>UserInformationException</i>). Server chủ động bắt lỗi và phản hồi dạng gói tin DTO thông báo lỗi rõ ràng về Client UI, tránh dùng Exception chung của Java gây lộ lọt thông tin hoặc làm sập luồng (thread crash).

3. Kỹ thuật quan trọng & Concurrency (Đồng thời)

Chức năng	Đảm bảo an toàn tuyệt đối khi có hàng ngàn yêu cầu đấu giá đồng thời cùng lúc vào một sản phẩm (tránh lost update, race condition). Hỗ trợ cập nhật giá theo thời gian thực (Realtime update) cho mọi client.
Giải pháp & Lý do	• Concurrency Control: Sử dụng cơ chế Pessimistic Locking (Khóa bi quan) qua DB bằng lệnh <code>SELECT ... FOR UPDATE</code> (trong hàm <i>findByIdForUpdate</i> của <i>AuctionDAO</i>) kết hợp quản lý Database Transaction thủ công (<code>setAutoCommit(false)</code>, <code>commit/rollback</code>). Buộc các yêu cầu trùng khít thời gian phải xếp hàng chờ ở tầng DB, ngăn chặn Lost Update và giữ dữ liệu tài chính luôn đồng nhất. • Cập nhật Realtime: Sử dụng TCP Sockets kết hợp EventBus. Khi một sự kiện <i>BidEvent</i> thành công, <i>BidEngine</i> phát sự kiện vào EventBus. Lớp <i>ClientNotificationObserver</i> ngay lập tức duyệt danh sách socket đang kết nối để broadcast dữ liệu mới về cho toàn bộ Client, triệt tiêu độ trễ của phương pháp Polling và tiết kiệm băng thông mạng.

4. Tích hợp, kiến trúc & Chất lượng mã

Chức năng	Hệ thống tổ chức theo mô hình Client-Server phân tán, tuân thủ kiến trúc MVC hai lớp độc lập, mã nguồn sạch quản lý bởi Maven, tích hợp Unit Test kiểm thử logic lỗi và luồng CI/CD tự động.
Giải pháp & Lý do	• Kiến trúc phân tán & MVC: Chia thành 3 module Maven (<i>client</i>, <i>server</i>, <i>common</i>). Client sử dụng JavaFX + FXML bóc tách giao diện khỏi Controller. Server xử lý ngầm qua chu trình Controller → Service/Engine → Model → DAO, đảm bảo bóc tách rành mạch vai trò của từng tầng logic. • Unit Test cho logic quan trọng: Sử dụng JUnit 5 kết hợp Mockito để viết kiểm thử cho <i>BidEngine</i> và <i>BidService</i>. Các kịch bản giả lập bao gồm: kiểm tra điều kiện trả giá hợp lệ, tính toán bước giá (bid increment) và kịch bản tranh chấp đồng thời. <i>Lý do:</i> Đảm bảo các hàm tính toán tài chính và nghiệp vụ cốt lõi hoạt động chính xác 100%, phát hiện sớm lỗi logic trước khi triển khai hệ thống. • Tự động hóa CI/CD: Thiết lập luồng GitHub Actions (<i>main.yml</i>). Mọi sự kiện Push/Pull Request đều tự động chạy lệnh <code>mvn clean test package</code> với JDK 25. Hệ thống sẽ tự kích hoạt toàn bộ các bộ Unit Test đã viết, nếu có bất kỳ test case nào thất bại, luồng build sẽ bị hủy, đảm bảo mã nguồn luôn ổn định.

5. Chức năng nâng cao (Điểm thưởng)

Chức năng	Tích hợp thành công 4 tính năng nghiệp vụ nâng cao phức tạp của đấu giá thực tế bao gồm: Auto-Bidding (Đấu giá tự động), Anti-sniping (Gia hạn giờ chốt), Trợ lý ảo AI thẩm định sản phẩm và Line Chart Realtime trực quan lịch sử giá.
Giải pháp & Lý do	• Auto-Bidding (Proxy Bidding): Triển khai tại <i>BidEngine.java</i> qua Thread Pool (<i>ExecutorService</i>). Khi giá thay đổi, luồng bất đồng bộ chạy nền quét thiết lập của <i>AutoBidDAO</i> và dùng thuật toán so sánh <i>maxBid</i> để tự động tăng giá thay mặt người dùng theo bước giá tối thiểu, giúp luồng chính Server không bị nghẽn. • Anti-sniping (Gia hạn thời gian): Trong hàm <i>executeBid()</i>, nếu thời gian còn lại của phiên ≤ 30 giây, hệ thống tự động kéo dài thời gian kết thúc thêm 1 phút (<code>endTime.plusMinutes(1)</code>). Giúp ngăn chặn hành vi dùng bot đặt giá ở giây cuối (sniper), mô phỏng quy trình búa đập truyền thống nhằm tối đa hóa lợi nhuận cho Seller. • Trợ lý ảo AI thẩm định: Kết nối OpenAI API qua thư viện <i>Langchain4j</i> trên nền Socket. Người dùng chat trực tiếp hoặc truyền ảnh sản phẩm (Base64) để nhờ AI thẩm định nguồn gốc, tính năng và định giá tham khảo nhanh chóng, áp dụng Generative AI tiên tiến nâng cao trải nghiệm người dùng. • Line Chart Realtime (Biểu đồ lịch sử): Sử dụng JavaFX <i>LineChart</i> kết hợp cơ chế Socket nhận dữ liệu từ Server. Client cập nhật biểu đồ an toàn bằng <code>Platform.runLater</code>, giúp dữ liệu hiển thị tức thời mà không cần refresh phòng đấu giá. Giúp Bidder quan sát xu hướng biến động và độ "nóng" của sản phẩm để đưa ra quyết định đặt giá hợp lý, đồng thời chứng minh năng lực xử lý đa luồng hiệu quả trong JavaFX.

PHẦN IV - PHÂN CÔNG NHIỆM VỤ NHÂN SỰ

Chi tiết trách nhiệm và hạng mục công việc cốt lõi của từng thành viên trong đội ngũ kỹ thuật

4.1. CHI TIẾT TRÁCH NHIỆM CÁ NHÂN

1. Trần Vũ Đức (Tech Leader)

Nhiệm vụ cốt lõi: Đảm nhận trọng trách chính ở mảng Client. Xây dựng và hoàn thiện toàn bộ giao diện người dùng (UI/UX) cũng như lập trình các tính năng trực tiếp tương tác với người dùng. Không chỉ giới hạn ở phía Frontend, Vũ Đức còn thể hiện tầm nhìn hệ thống tốt khi chủ động hỗ trợ nhóm Backend định hình lại kiến trúc tập tin và refactor mã nguồn Server, cùng với đó là thiết kế networking cùng với model AI Local, giúp cấu trúc code trở nên rõ ràng và dễ bảo trì hơn, đồng thời giúp dự án có thêm tính năng hữu ích.

2. Lê Nguyên Chương (Core Developer)

Nhiệm vụ cốt lõi: Là nhân sự chủ chốt phía Backend, chịu trách nhiệm thiết kế hệ thống Cơ sở dữ liệu (Database) và xây dựng nền tảng vững chắc cho các lớp xử lý logic. Phối hợp chặt chẽ với Vũ Thành Đạt để hoàn thiện toàn bộ luồng xử lý của Server. Ngoài ra, Chương cũng đóng góp các ý tưởng và trực tiếp hỗ trợ Vũ Đức trong công tác thiết kế giao diện Client.

3. Vũ Thành Đạt (Core Developer)

Nhiệm vụ cốt lõi: Chịu trách nhiệm thiết lập luồng giao tiếp mượt mà và theo thời gian thực giữa Client và Server thông qua nền tảng Socket. Quản lý và xây dựng các Controller điều hướng dữ liệu. Hỗ trợ Chương xử lý các tác vụ phức tạp phía Server. Bên cạnh đó, Đạt cũng trực tiếp tham gia cùng Minh Đức trong khâu kiểm thử phần mềm để đảm bảo đường truyền và dữ liệu không bị nghẽn hay mất mát.

4. Lê Minh Đức (Core Developer)

Nhiệm vụ cốt lõi: Chuyên trách mảng Đảm bảo chất lượng (QA). Thực hiện ứng dụng các mẫu thiết kế hướng đối tượng (điển hình là Observer Pattern) vào kiến trúc phần mềm, đồng thời lên kịch bản kiểm thử (Test). Rà soát mã nguồn tổng thể để dự đoán và thiết lập các bộ xử lý ngoại lệ (Exception handling), giúp hệ thống tránh khỏi tình trạng sập đột ngột khi vận hành thực tế.

THÀNH VIÊN	PHÂN ĐOẠN CHÍNH	SẢN PHẨM ĐẦU RA DỰ KIẾN
Trần Vũ Đức	Client UI/UX & AI	Mã nguồn Frontend, Model AI Local, thiết kế mạng.
Lê Nguyên Chương	Database & Core BE	Sơ đồ ERD, hệ quản trị dữ liệu, API Endpoints.
Vũ Thành Đạt	Socket & Real-time	Bộ Controller điều phối, hệ thống Socket Gateways.
Lê Minh Đức	QA & Testing	Kịch bản kiểm thử, tài liệu xử lý ngoại lệ hệ thống.

* **Ghi chú:** Nhân sự chủ động phối hợp chéo theo cặp (Vũ Đức - Nguyên Chương; Nguyên Chương - Thành Đạt; Thành Đạt - Minh Đức) nhằm đảm bảo tiến độ và tính nhất quán dữ liệu toàn hệ thống.